UTampa Parking Sprint Planning

Team Name: OpenSpot

Team Members: Sayed, Jack, Dhwani, Khalil, Anthony, Lailani

GitHub repository: https://github.com/imranali17689/Sensors.git

Trello board: https://trello.com/b/ktlaY3uY/my-trello-board?completedInviteSignup=1

Sprint 1: Detailed Sprint

Backend Software Developer: Khalil Smith

Sprint Goal: Set up the foundation for the FastAPI backend and prove the backend is receiving a sensor update to store in Supabase and return current spot status to the frontend.

Critical Tasks:

1. Backend project infrastructure
2. Supabase database integration
3. Sensor ingestion endpoint
4. Frontend fetch endpoint
5. Basic validation + counting
6. Testing + API documentation

Tasks:

1. Initialize FastAPI project structure (2 hrs)
2. Connect to Supabase (client module + test query)
3. POST (/sensor-update) - updating and inserting the logs
4. GET (/spots) - return the status

Dependencies:

1. Supabase connection points to the database
2. Frontend to display the data from the database and sensors

Completed For Sprint 1:

1. Start FastAPI locally
2. Supabase connection
3. Call POST/ garage-event
4. Log events in the database
5. Call GET /status returning current totals

Frontend Software Developer: Sayed Ali

Sprint goal: Build the frontend foundation for OpenSpot by creating the core user interface,including a dashboard, and connect it to a basic API endpoint (or mock data) to display real-time parking availability.

1. Frontend project setup (React + folder structure + basic styling)
2. Build main dashboard UI (total spots, available spots, status display)
3. Implement different parking garages views
4. Integrate API fetch (or mock data if backend not ready)
5. Basic navigation between pages (dashboard + parking view)
6. UI styling and layout improvements for demo readiness

Effort estimation low, should be able to have a basic UI ready in a week or 2. Goal is to get it set up and connect to the backend once superbase is setup.

Database Developer: Anthony Eccleston

Sprint Goal

Design and implement the Supabase database schema that supports:

- Sensor ingestion
- Real-time counting
- Data validation
- Historical tracking

Critical Tasks (Must Be Done Early)

Week 1 (High Priority – Enables Backend)

- Design ER structure
- Create tables in Supabase:
    - garage

    - sensors
    - sensor_events
    - Garage_state
- Add constraints:
    - No negative counts
    - No exceeding capacity
    - Direction limited to IN/OUT
- Insert seed data (1 garage, 2 sensors)

Week 2 (Stability + Optimization)

- Add indexes
- Test concurrent update behavior
- Verify counting logic works with backend
- Support analytics queries
- Document schema

Hardware systems developer: Jack + Lailani

Sprint Goal: ESP32 can reliably detect vehicle entry/exit events using ultrasonic sensors and successfully transmit valid data to the backend.

Critical Tasks:

1. Wire ESP32-D with Ultrasonic sensors (A+B)
2. Implement 5v -> 3.3v on echo pin (esp32 pins are rated for 3.3v)
3. Validate stable distance readings
4. Implement logic: A→B = Entry, B→A = Exit
5. Connect ESP32 to wifi/ send Post events
6. Research solar requirements

Effort estimation medium, hard tasks include tuning detection accuracy, preventing false triggers, and ensuring reliable wifi communication. Will likely need few iterations of debugging with hardware and logic.

Sprint 2: Overview

Backend Software Developer:

- Handle real sensor data (not mock)
- Improve ingestion endpoint reliability
- Add real-time update logic
- Handle edge cases (duplicate triggers, missed events)
- Implement official occupied + available spot calculation
- Enforce garage capacity limit based on the 4th and 5th floor
- Prevent:
    - Negative counts
    - Exceeding total capacity
- Improve validation for enter/exit events

Frontend Software Developer:

- Replace mock data with real API data
- Show live updates (auto refresh)
- Improve UI clarity, Per garage view
- Add loading + error states

Database Developer:

- Ensure accurate counting under load
- Fix race conditions (multiple sensors hitting at once)
- Optimize queries for fast reads
- Validate constraints actually work

Hardware systems developer:

- Improve detection accuracy
- Reduce false positives
- Ensure consistent WiFi transmission

- Test multiple real-world scenarios

Accuracy & Physical Design-

Refine detection reliability and implement stable counting logic. Begin 3D enclosure design and determine mounting strategy for garage deployment.

Sprint 3: Overview

Backend Software Developer:

- Integrate actual entrance/exit sensors
- Replace simulated events with real hardware events
- Add real-time updates (Supabase Realtime)
- Ensure frontend updates instantly when count changes
- Handle rapid consecutive events safely

Frontend Software Developer:

Consider any extra UI improvements we can make, including trends, usage patterns, etc..

Database Developer:

- Support historical analytics queries
- Optimize for scaling (more sensors, more garages)
- Clean up schema if needed

Hardware systems developer:

- Deploy across multiple sensors

- Stress test system (rapid entries/exits)
- Improve durability + placement

Sprint 4: Overview

Turn OpenSpot into a clean, demo-ready product

Backend Software Developer:

- Improve logging and monitoring
- Optimize database queries
- Strengthen error handling
- Final API cleanup and documentation
- Stress test system with multiple rapid events

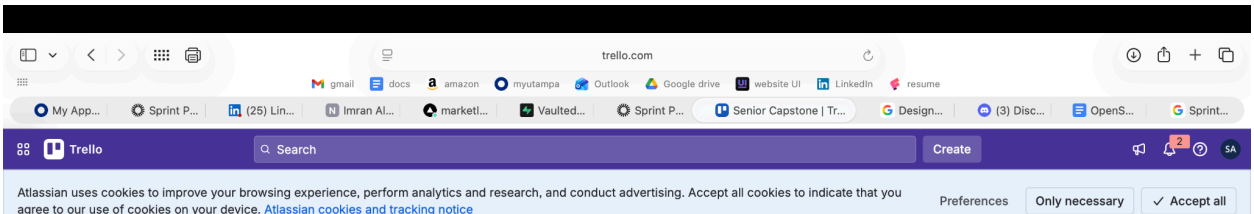
Frontend Software Developer:

Should be completely finished with the frontend by now, but any last minute polishes and animations

Database Developer:

- Final cleanup + documentation
- Ensure
- consistency

Hardware systems developer:

- Clean physical setup
- Reliable demo (no glitches)

Trello Screenshots: Sprint Planning